Related Work & Solution Approach

Two Recent Papers on Payment Failure Detection and Fraud Risk — Mapped to Our Problem Statement

1. Our Problem Statement, Restated

We are building a simulated UPI payment platform. Underneath the send-money / track / history interface, the system runs concurrent fraud-risk checks, tracks payment state end-to-end, prevents duplicate payments via idempotency, and automatically reconciles cases where the sender is debited but the receiver is not credited. For stuck transactions, the system estimates a probable recovery date/time and issues the right alert and guidance. An ops dashboard monitors suspicious transactions, reconciliation cases, alerts, and logs.

This splits cleanly into two sub-problems, and the two papers below map onto them one-to-one: **(A) is this transaction fraudulent?** and **(B) why is this transaction stuck, and what do we do about it?**

2. Paper 1 — The Fraud Sub-Problem

Fraud Detection in UPI Payments Using Tabular Machine Learning Models

Chaudhary, Singh, Singh, Zaidi, Jain • IJRASET, Vol. 13, Issue X • October 2025 • DOI: 10.22214/ijraset.2025.74922

What is the problem?

As UPI transaction volume grows, fraud grows with it, but there is no public UPI transaction dataset to train a detector on, and UPI data is heavily categorical (device ID, merchant category, channel) and extremely imbalanced — real fraud is a small fraction of a percent of all transactions.

Why does this happen?

Fraud detection is fundamentally a statistical pattern-recognition problem: a transaction looks fraudulent because of how it compares to thousands of other transactions' behavior (amount, timing, device, velocity), not because of a single deterministic rule. That need for comparison against history is exactly why rule-based checks alone eventually fall short here, and exactly why no real UPI dataset exists publicly — banks don't release transaction-level data, so every academic study (including ours) has to simulate one.

How does this relate to our problem statement?

This is a direct match to the fraud-risk-check module in our own system. We face the identical dataset problem they do (no real UPI data), and the identical data shape (categorical, imbalanced). Their reasoning for model choice is directly reusable as justification for our own fraud layer.

How did they solve it?

- Generated a synthetic dataset — 100,000 transactions, 30,000 senders/receivers, 25 features (identification, transaction, behavioral, risk-context), with a realistic 0.63% fraud rate.
- Trained and compared two gradient-boosting models: **CatBoost** and **XGBoost**, on an 80:20 stratified split.
- Deployed the trained model behind a real-time Streamlit app (“Quicki”) that scores a transaction's fraud probability live.

Why that method, and not the alternatives?

- **Not Logistic Regression** — too simple for the non-linear, high-dimensional relationships in transaction data; underfits.

- **Not plain Random Forest / XGBoost without care** — both need categorical features manually encoded first; one-hot encoding on high-cardinality fields like Device_ID blows up the feature space, and label encoding introduces bias.
- **Not Deep Learning / NLP models** — flagged as an over-used trend in prior literature, despite structured tabular financial data not benefiting much from it.
- **CatBoost specifically** — handles categorical variables natively via ordered boosting and target-based encoding (avoiding the information leakage naive target encoding causes), and is more robust under severe class imbalance.

Result: CatBoost beat XGBoost on ROC-AUC (0.870 vs 0.844) and recall (0.60 vs 0.42) — catching more real fraud — at the cost of slightly lower precision (0.49 vs 0.62). They judged recall more important here, since missing real fraud costs more than a false alarm.

3. Paper 2 — The Failure-Diagnosis Sub-Problem

A Feature Engineering Approach for Business Impact-Oriented Failure Detection in Distributed Instant Payment Systems

Lorenzo Porcelli • Bank of Italy / University of Salerno • arXiv:2510.21710 • last revised August 2025

What is the problem?

TIPS (TARGET Instant Payment Settlement) — the real EU-wide instant payment infrastructure — must settle every payment within 5 seconds with zero downtime. No single participant (sender bank, receiver bank, clearing mechanism) sees the whole transaction; each only observes the messages that pass through it. Traditional monitoring watches each component in isolation, so when the fault lies between organizations, every individual component can look perfectly healthy while the end-to-end payment is actually failing.

Why does this happen?

This is structurally the same distributed-systems gap behind our own problem statement: the debit side and credit side are owned and monitored by different systems, connected only by an unreliable network, so nobody has a single authoritative view of “what actually happened” to a given transaction in real time.

How does this relate to our problem statement?

This is a direct, production-grade precedent for our reconciliation/diagnosis engine — the module that figures out *why* a transaction is stuck (debited-not-credited, timeout, network failure) and localizes the likely cause. Their internal-vs-external failure localization is conceptually identical to our sender-side-vs-receiver-side diagnosis.

How did they solve it?

- Engineered just **3 timing features** from standard ISO 20022 message timestamps every instant-payment system already exchanges: d1 (time inside the clearing mechanism before forwarding), d2 (time waiting on the receiver bank's reply), d3 (time to finalize settlement) — plus settled-payment volume as a 4th signal.
- Ran an off-the-shelf, algorithm-agnostic **anomaly detector** over this small, compact feature set to flag deviations from normal behavior.
- Applied a **simple rule-based explainer** on top: if d1 or d3 spike without d2 spiking -> internal failure; if only d2 spikes -> external failure. Severity is similarly rule-mapped from anomaly-score thresholds.

Why that method, and not the alternatives?

- **Not deep learning on raw logs** (LSTM/Transformer/GNN) — rejected for two reasons: real-time latency makes post-hoc explainability tools (SHAP/LIME) too slow on high-frequency streams, and a black-box model flags “something's wrong” without saying where or why.
- **Not raw infrastructure metrics** — these only show component-level health, missing failures that occur strictly between organizations (the exact gap this paper targets).
- **Hand-engineered, interpretable features instead** — each feature maps to one specific transaction phase by design, so the moment one is anomalous, the cause is already localized without needing a separate explanation step.
- **A rule-based explainer instead of a second ML classifier** — for the same reason our own diagnosis engine uses rules: strong domain knowledge of exactly which signal pattern means what makes a transparent rule more trustworthy and auditable in a regulated financial system than a model nobody can interrogate.

Result: validated on a real August 2023 TIPS incident (a network-provider outage that cut settled volume by 78%) — correctly localized as external, correctly classified as critical — plus four controlled stress-injection scenarios, all correctly localized and severity-ranked.

4. Side-by-Side: How Both Papers Justify Our Two-Engine Design

	Fraud Paper (CatBoost)	Failure Paper (Feature+Rules)	What it confirms for us
Core question	Is this transaction fraudulent?	Why is this transaction stuck, and where is the fault?	Two genuinely different questions -- don't force one method to answer both.
Nature of the problem	Statistical pattern recognition over many past examples	Deterministic diagnosis from real-time signals, no history needed	Method should match problem type, not project preference.
Chosen method	ML (CatBoost, gradient boosting)	Feature engineering + rule-based explainer	Our split (rules for diagnosis, ML for fraud) mirrors both papers.
Why not deep learning	Overkill for structured tabular data; prior work over-used it	Too slow for real-time; not explainable without extra tooling	Avoid ML (esp. deep learning) wherever a simpler method is auditable and fast enough.
Data source	Synthetic (Faker-generated), since no public UPI dataset exists	Real production data (TIPS) + controlled synthetic injection	Synthetic data is an accepted, defensible choice at our scale -- pair it with our own simulator's logs where possible.
Explainability stance	Feature importances surfaced post-hoc for trust/regulation	Explainability built into the features themselves, by design	Prefer designing diagnosis signals to be self-explanatory over bolting on explanation later.

5. How We Should Approach Building Our Own Solution

5.1 Keep the two engines separate, and justify each independently

Don't build one model that tries to decide fraud risk and payment state together. Our rule engine (payment state machine, timeout detection, debit/credit mismatch, idempotency) should look like Paper 2's approach: small, interpretable, real-time signals and transparent if-then rules, not a trained model. Our fraud layer should look like Paper 1's approach: a tabular gradient-boosting model trained on engineered behavioral features, evaluated with recall/precision/ROC-AUC rather than plain accuracy, given how imbalanced fraud data always is.

5.2 Design our diagnosis features the way Paper 2 does, not the way we might default to

Rather than dumping every available field into a black box, define a small number of timing/state features tied to specific phases of our own payment flow (e.g. time-to-debit-confirmation, time waiting on receiver response, time-to-settlement-confirmation) -- mirroring d1/d2/d3. This gives us the same free localization benefit: a spike in the "waiting on receiver" feature alone already tells us the fault is external, no extra model needed.

5.3 Build our dataset strategy the way both papers do

Neither paper had access to real proprietary transaction data, and neither apologized for that -- they stated the limitation plainly and designed around it. We should do the same: generate our own synthetic transaction stream (our simulator's own logs double as training data for recovery-time estimation, matching Paper 1's approach), and be explicit in our report that this proves the concept rather than claiming production-scale validation.

5.4 Reserve ML for genuinely statistical sub-questions only

Fraud scoring and recovery-time estimation are legitimate ML use cases (they need historical patterns). Payment-state tracking, duplicate detection, and real-time fault localization are not -- they are solvable with deterministic logic, and both papers show that forcing ML into those parts would mean slower, less explainable, and less trustworthy answers for no accuracy benefit.

5.5 Treat explainability as a design constraint, not an afterthought

Paper 2's strongest lesson: don't build an opaque detector and then try to explain it after the fact. Design signals and rules so the explanation is built into the output from the start -- this is exactly what our ops dashboard needs to show (why a transaction is flagged, not just that it is).

References: (1) R. Chaudhary, S. Singh, R. Singh, H. Zaidi, K. Jain, "Fraud Detection in UPI Payments Using Tabular Machine Learning Models," IJRASET, Vol. 13, Issue X, Oct 2025, doi:10.22214/ijraset.2025.74922. (2) L. Porcelli, "A Feature Engineering Approach for Business Impact-Oriented Failure Detection in Distributed Instant Payment Systems," arXiv:2510.21710, last revised Aug 2025.